

Chapter 21. Conclusions and where next for Java

This chapter covers

- The new Java 8 features and their evolutionary effect on programming style
- The new Java 9 module system
- The new six-monthly Java incremental-release life cycle
- The first incremental release forming Java 10
- A few ideas that you'll likely see implemented in some future version of Java

We've covered a lot of material in this book, and we hope that you feel that you're ready to start using the new Java 8 and 9 features in your own code, perhaps building on our examples and quizzes. In this chapter, we review the journey of learning about Java 8 and the gentle push toward functional-style programming, as well as the advantages of the new modularization capability and other minor improvements introduced with Java 9. You also learn about what is included in Java 10. In addition, we speculate on what future enhancements and great new features may be in Java's pipeline beyond Java 9, 10, 11, and 12.

21.1. Review of Java 8 features

A good way to help you understand Java 8 as a practical, useful language is to revisit the features in turn. Instead of simply listing them, we'd like to present them as being interlinked to help you understand them not only as a set of features, but also as a high-level overview of the coherent language design that is Java 8. Our other aim in this review chapter is to emphasize how most of the new features in Java 8 are facilitating functional-style programming in Java. Remember, supporting function pro-

Other formats available



Audiobook [↗](#)

✗ A conscious design choice but a conscious design strategy, which we regard as climate change in the

- The increasing need to exploit the power of multicore processors now that, for silicon technology reasons, the additional transistors annually provided by Moore's law no longer translate into higher clock speeds

of individual CPU cores. Put simply, making your code run faster requires parallel code.

- The increasing tendency to concisely manipulate collections of data with a declarative style for processing data, such as taking some data source, extracting all data that matches a given criterion, and applying some operation to the result (summarizing it or making a collection of the result for further processing later). This style is associated with the use of immutable objects and collections, which are then processed to produce further immutable values.

Neither motivation is effectively supported by the traditional, object-oriented, imperative approach, which centers on mutating fields and applying iterators. Mutating data on one core and reading it from another is surprisingly expensive, not to mention that it introduces the need for error-prone locking. Similarly, when you're focused on iterating over and mutating existing objects, the stream-like programming idiom can feel alien. But these two trends are supported by ideas from functional programming, which explains why the Java 8 center of gravity has moved a bit from what you've come to expect from Java.

This chapter reviews, in a big-picture unifying view, what you've learned from this book and shows you how everything fits together in the new climate.

21.1.1. Behavior parameterization (lambdas and method references)

To write a reusable method such as `filter`, you need to specify as its argument a description of the filtering criterion. Although Java experts could achieve this task in earlier versions of Java by wrapping the filtering criterion as a method inside a class and passing an instance of that class, this solution was unsuitable for general use because it was too cumbersome to write and maintain.

As you discovered in [chapters 2](#) and [3](#), Java 8 provides a way, borrowed from functional programming, to pass a piece of code to a method. Java conveniently provides two variants:

Other formats available



Audiobook



(e.g., a piece of code) such as

```
apple -> apple.getWeight() > 150
```

- Passing a *method reference* to an existing method, such as `Apple::isHeavy`

These values have types such as `Function<T, R>`, `Predicate<T>`, and `BiFunction<T, U, R>`, and the recipient can execute them by using the methods `apply`, `test`, and so on. These types are called functional interfaces and have a single abstract method, as you learned in [chapter 3](#). Of themselves, lambdas can seem to be rather a niche concept, but the way that Java 8 uses them in much of the new Streams API propels them to the center of Java.

21.1.2. Streams

The collection classes in Java, along with iterators and the `for-each` construct, have served programmers honorably for a long time. It would have been easy for the Java 8 designers to add methods such as `filter` and `map` to collections, exploiting lambdas to express database-like queries. Instead, the designers they added a new Streams API, which is the subject of [chapters 4–7](#), and it's worth pausing to consider why.

What's wrong with `Collections` that requires them to be replaced or augmented by a similar notion of `Streams`? We'll summarize this way: if you have a large collection and apply three operations to it (perhaps mapping the objects in the collection to sum two of their fields, filtering the sums that satisfy some criterion, and sorting the result), you make three separate traversals of the collection. Instead, the Streams API lazily forms these operations into a pipeline and does a single stream traversal performing all the operations together. This process is much more efficient for large datasets, and for reasons such as memory caches, the larger the dataset, the more important it is to minimize the number of traversals.

The other, no less important, reason concerns processing elements in parallel, which is vital to the efficient exploitation of multicore CPUs.

Streams, and in particular the `parallel` method, allow a stream to be marked as suitable for parallel processing. Recall that parallelism and

Other formats available



Audiobook



...other, so core functional concepts (side-effect-free, stateless, parameterized with lambdas and method references) are central to exploiting streams in parallel by using `map`, `filter`, and the like.

In the following section, we look at how these ideas, which we introduced in terms of streams, have a direct analog in the design of

CompletableFuture.

21.1.3. CompletableFuture

Java has provided the `Future` interface since Java 5. Futures are useful for exploiting multicore because they allow a task to be spawned onto another thread or core and allow the spawning task to continue executing along with the spawned task. When the spawning task needs the result, it can use the `get` method to wait for the `Future` to complete (produce its value).

Chapter 16 explains the Java 8 `CompletableFuture` implementation of `Future`, which again exploits lambdas. A useful, if slightly imprecise, motto is “`CompletableFuture` is to `Future` as `Stream` is to `Collection`.” To compare:

- `Stream` lets you pipeline operations and provides behavior parameterization with `map`, `filter`, and the like, eliminating the boilerplate code that you typically have to write when you use iterators.
- `CompletableFuture` provides operations such as `thenCompose`, `thenCombine`, and `allOf`, which provide functional-programming-style concise encodings of common design patterns involving `Futures` and let you avoid similar imperative-style boilerplate code.

This style of operations, albeit in a simpler scenario, also applies to the Java 8 operations on `Optional`, which we revisit in the next section.

21.1.4. Optional

The Java 8 library provides the class `Optional<T>`, which allows your code to specify that a value is a proper value of type `T` or a missing value returned by the static method `Optional.empty`. This feature is great for program comprehension and documentation. It provides a data type with an explicit missing value instead of the previous error-prone use of the `null` pointer to indicate missing values, which programmers could never be sure was a planned missing value or an accidental `null` resulting from a computation.

Other formats available



Audiobook

if `Optional<T>` is used consistently, programs should never produce `NullPointerExceptions`. Again, you could see this situation as a one-off, unrelated to the rest of Java 8, and ask, “How does changing from one form of missing value to another help me write programs?” Closer inspection shows that the `Optional<T>` class

provides `map`, `filter`, and `ifPresent`. These methods have behavior similar to that of corresponding methods in the `Streams` class and can be used to chain computations, again in functional style, with the tests for missing value done by the library instead of user code. The choice of internal versus external testing in `Optional<T>` is directly analogous to how the `Streams` library does internal versus external iteration in user code. Java 9 added various new methods to the `Optional` API, including `stream()`, `or()`, and `ifPresentOrElse()`.

21.1.5. Flow API

Java 9 standardized reactive streams and the reactive-pull-based back-pressure protocol, a mechanism designed to prevent a slow consumer from being overwhelmed by one or more faster producers. The Flow API includes four core interfaces that library implementations can support to provide wider compatibility: `Publisher`, `Subscriber`, `Subscription`, and `Processor`.

Our final topic in this section concerns not functional-style programming, but Java 8 support for upward-compatible library extensions driven by software-engineering desires.

21.1.6. Default methods

Java 8 has other additions, none of which particularly affects the expressiveness of any program. But one thing that's helpful to library designers allows default methods to be added to an interface. Before Java 8, interfaces defined method signatures; now they can also provide default implementations for methods that the interface designer suspects not all clients will want to provide explicitly.

This tool is a great new tool for library designers because it gives them the ability to augment an interface with a new operation without having to require all clients (classes implementing this interface) to add code to define this method. Therefore, default methods are also relevant to users of libraries because they shield the users from future interface changes

Other formats available



Audiobook



e system

Java 8 added a lot, both in terms of new features (lambdas and default methods on interfaces, for example) and new useful classes in the native API, such as `Stream` and `CompletableFuture`. Java 9 didn't introduce

any new language features but mostly polished the work started in Java 8, completing the classes introduced there with some useful methods such as `takeWhile` and `dropWhile` on a `Stream` and `completeOnTimeout` on a `CompletableFuture`. In fact, the main focus of Java 9 was the introduction of the new module system. This new system doesn't affect the language except for the new `module-info.java` file, but nevertheless improves the way in which you design and write applications from an architectural point of view, clearly marking the boundaries of subparts and defining how they interact.

Java 9, unfortunately, harmed the backward compatibility of Java more than any other release (try compiling a large Java 8 code base with Java 9). But this cost is worth paying for the benefits of proper modularization. One reason is to ensure better and stronger encapsulation across packages. In fact, Java visibility modifiers are designed to define encapsulation among methods and classes, but across packages, only one visibility is possible: `public`. This lack makes it hard to modularize a system properly, in particular to specify which parts of a module are designed for public use and which parts are implementation details that should be hidden from other modules and applications.

The second reason, which is an immediate consequence of the weak encapsulation across packages, is that without a proper module system, it's impossible to avoid exposing functionalities that are relevant for security of all the code running in the same environment. Malicious code may access critical parts of your module, thus bypassing all security measures encoded in them.

Finally, the new Java module system enables the Java runtime to be split into smaller parts, so you can use only the parts that are necessary for your application. It would be surprising if CORBA was a requirement for your new Java project, for example, but it's likely to be included in all your Java applications. Although this act may be of limited relevance for traditional-size computing devices, it's important for embedded appliances and for the increasingly frequent situation in which your Java ap-

×erized environment. In other words, the Java
er that allows the use of the Java runtime in
lications and in the cloud.

Other formats available



As discussed in [chapter 14](#), the Java module system solves these problems by introducing a language-level mechanism to modularize your large systems and the Java runtime itself. The advantages of the Java module system include the following:

- **Reliable configuration**— Explicitly declaring module requirements allows early detection of errors at build time rather than at runtime in the case of missing, conflicting, or circular dependencies.
- **Strong encapsulation**— The Java Module System enables modules to export only specific packages and then separate the public and accessible boundaries of each module with internal implementation.
- **Improved security**— Not allowing users to invoke specific parts of your module makes it much harder for an attacker to evade the security controls implemented in them.
- **Better performance**— Many optimization techniques can be more effective when a class can refer to few components rather than to any other classes loaded by the runtime.
- **Scalability**— The Java module system allows the Java SE platform to be decomposed into smaller parts containing only the features required by the running application.

In general, modularization is a hard topic, and it's unlikely to be a driver for quick adoption of Java 9 as lambdas were for Java 8. We believe, however, that in the long run, the effort you invest in modularizing your application will be repaid in terms of easier maintainability.

So far, we've summarized the concepts of Java 8 and 9 covered in this book. In the next section, we turn to the thornier subject of future enhancements and great features that may be in Java's pipeline beyond Java 9.

21.3. Java 10 local variable type inference

Originally in Java, whenever you introduced a variable or method, you gave its type at the same time. The example

```
double convertUSDToGBP(double money) { ExchangeRate e = ...; }
```

contains three types, which give the result the type of `convertUSDToGBP`, the type of its argument `money`, and the type of its local variable `e`. Over

Other formats available



Audiobook



been relaxed in two ways. First, you may omit types in an expression when the context deter-

```
Map<String, List<String>> myMap = new HashMap<String, List<String>>
```


can be abbreviated to the following since Java 7:

```
Map<String, List<String>> myMap = new HashMap<>();
```

Second, to use the same idea of propagating the type determined by context into an expression, a lambda expression such as

```
Function<Integer, Boolean> p = (Integer x) -> booleanExpression;
```

can be shortened to

```
Function<Integer, Boolean> p = x -> booleanExpression;
```

by omitting types. In both cases, the compiler infers the omitted types.

Type inference has a few advantages when a type consists of a single identifier, the main one being reduced editing work when replacing one type with another. But as types grow in size, generics parameterized by further generic types, type inference can aid readability.^[1] The Scala and C# languages permit a type in a local-variable-initialized declaration to be replaced by the (restricted) keyword `var`; the compiler fills in the appropriate type from the right side. The declaration of `myMap` shown earlier in Java syntax could be rendered like this:

¹

It's important that type inference be done sensibly, of course. Type inference works best when you have only one way, or one easily documentable way, to re-create the type that the user omitted. Problems occur if the system infers a different type from the one that the user was thinking of. So a good design of type inference produces a fault when two incomparable types could be inferred; heuristics can give the appearance of picking the wrong one seemingly at random.

Other formats available

 Audiobook 

✕ `HashMap<String, List<String>>();`

variable type inference and is included in Java 10.

There's some small cause for concern, however. Given a class `Car` that subclasses a class `Vehicle`, does the declaration


```
var x = new Car();
```

implicitly declare `x` to have type `Car` or `Vehicle` (or even `Object`)? In this case, a simple explanation that the missing type is the type of the initializer (here, `Car`) is perfectly clear. Java 10 formalizes this fact, also stating that `var` can't be used when there's no initializer.

21.4. What's ahead for Java?

Some of the points we cover in this section are discussed in more detail on the JDK Enhancement Proposal website at

<http://openjdk.java.net/jeps/0>. Here, we take care to explain why seemingly sensible ideas have subtle difficulties or interactions with existing features that inhibit their direct incorporation into Java.

21.4.1. Declaration-site variance

Java supports wildcards as flexible mechanisms that allow subtyping for generics (generally referred to as *use-site variance*). This support makes the following assignment valid:

```
List<? extends Number> numbers = new ArrayList<Integer>();
```

But the following assignment, omitting the `"? extends"`, produces a compile-time error:

```
List<Number> numbers = new ArrayList<Integer>();
```

1

- **1 Incompatible types**

Many programming languages, such as C# and Scala, support a different variance mechanism called *declaration-site variance*. These languages allow programmers to specify variance when defining a generic class. This

that are inherently variant. `Iterator`, for example, is inherently covariant, and `Comparator` is inherently contravariant. Thinking in terms of `? extends` or `? super` for declaration-site variance to Java would be

useful, because these specifications appear instead at the declaration of classes. As a result, this addition would reduce some cognitive overhead for programmers. Note that at the writing (2018), a JDK enhancement pro-

Other formats available

 Audiobook 

positional would allow default declaration-site variance in upcoming versions of Java (<http://openjdk.java.net/jeps/300>).

21.4.2. Pattern matching

As we discussed in [chapter 19](#), functional-style languages typically provide some form of pattern matching—an enhanced form of `switch`—in which you can ask, “Is this value an instance of a given class?” and (optionally) recursively ask whether its fields have certain values. In Java a simple case test looks like this:

```
if (op instanceof BinOp){
    Expr e = ((BinOp) op).getLeft();
}
```

Note that you have to repeat the type `BinOp` within the cast expression, even though it’s clear that the object referenced by `op` is of that type.

You may have a complicated hierarchy of expressions to process, of course, and the approach of chaining multiple `if` conditions will make your code more verbose. It’s worth reminding you that traditional object-oriented design discourages the use of `switch` and encourages patterns such as the visitor pattern, in which data-type-dependent control flow is done by method dispatch instead of by `switch`. At the other end of the programming language spectrum, in functional-style programming, pattern matching over values of data types is often the most convenient way to design a program.

Adding Scala-style pattern matching in full generality to Java seems to be a big job, but following the recent generalization to `switch` to allow `Strings`, you can imagine a more modest syntax extension that allows `switch` to operate on objects by using the `instanceof` syntax. In fact, a JDK enhanced proposal explores pattern matching as a language feature for Java (<http://openjdk.java.net/jeps/305>). The following example revisits our example from [chapter 19](#) and assumes a class `Expr`, which is sub-

Other formats available

 Audiobook 

Number:

```
case (op instanceof BinOp):
    doSomething(op.getOpName(), op.getLeft(), op.getRight());
case (n instanceof Number):
    dealWithLeafNode(n.getValue());
default:
```

```
        defaultAction(someExpr);  
    }  
}
```

Notice a couple of things. First, this code steals from pattern matching the idea that in `case (op instanceof BinOp):`, `op` is a new local variable (of type `BinOp`), which becomes bound to the same value as `someExpr`. Similarly, in the `Number` case, `n` becomes a variable of type `Number`. In the default case, no variable is bound. This proposal eliminates much boilerplate code compared with using chains of `if-then-else` and casting to subtype. A traditional object-oriented designer probably would argue that such data-type dispatch code would better be expressed with visitor-style methods overridden in subtypes, but to functional-programming eyes, this solution results in related code being scattered over several class definitions. This classical design dichotomy is discussed in the literature as the expression problem.^[2]

²

For a more complete explanation, see
http://en.wikipedia.org/wiki/Expression_problem.

21.4.3. Richer forms of generics

This section discusses two limitations of Java generics and looks at a possible evolution to mitigate them.

Reified generics

When generics were introduced in Java 5, they had to be backward-compatible with the existing JVM. To this end, the runtime representations of `ArrayList<String>` and `ArrayList<Integer>` are identical. This model is called the *erasure model of generic polymorphism*. Certain small runtime costs are associated with this choice, but the most significant effect for programmers is that parameters of generic types can be only objects and not primitive types. Suppose that Java allowed, say, `ArrayList<int>`. Then you could allocate an `ArrayList` object on the

Other formats available

 Audiobook 

× the value such as `int 42`, but the `ArrayList` has no indicator of whether it contained an object or a primitive `int` value such as `42`.

At some level, this situation seems to be harmless. If you get a primitive `42` from an `ArrayList<int>` and a `String` object `"abc"` from an `ArrayList<String>`, why should you worry that the `ArrayList` con-

tainers are indistinguishable? Unfortunately, the answer is garbage collection, because the absence of runtime type information about the contents of the `ArrayList` would leave the JVM unable to determine whether element 13 of your `ArrayList` was a `String` reference (to be followed and marked as in use by garbage collection) or an `int` primitive value (most definitely not to be followed).

In the C# language, the runtime representations of `ArrayList<String>`, `ArrayList<Integer>`, and `ArrayList<int>` are in principle different. But even if these representations were the same, sufficient type information is kept at runtime to allow, for example, garbage collection to determine whether a field is a reference or a primitive. This model is called the *reified model of generic polymorphism* or, more simply, *reified generics*. The word *reification* means “making explicit something that otherwise would be implicit.”

Reified generics are clearly desirable because they enable a more full unification of primitive types and their corresponding object types—something that you’ll see as problematic in the following sections. The main difficulty for Java is backward compatibility, both in the JVM and in existing programs that use reflection and expect generics to be erased.

Additional syntactic flexibility in generics for function types

Generics proved to be a wonderful feature when they were added to Java 5. They’re also fine for expressing the type of many Java 8 lambdas and method references. You can express a one-argument function this way:

```
Function<Integer, Integer> square = x -> x * x;
```

If you have a two-argument function, you use the type `BiFunction<T, U, R>`, where `T` is the type of the first parameter, `U` the second, and `R` the result. But there’s no `TriFunction` unless you declare it yourself.

Similarly, you can’t use `Function<T, R>` for references to methods that

Other formats available

 Audiobook 

✕ return result type `R`; you have to use

In essence, Java 8 lambdas have enriched what you can write, but the type system hasn’t kept up with the flexibility of the code. In many functional languages, you can write, for example, the type `(Integer, Double) => String`, to represent what Java 8 calls

`BiFunction<Integer, Double, String>`, along with `Integer => String` to represent `Function<Integer, String>`, and even `() => String` to represent `Supplier<String>`. You can understand `=>` as an infix version of `Function`, `BiFunction`, `Supplier`, and the like. A simple extension to Java syntax for types to allow infix `=>` would result in more readable types analogous to what Scala provides, as discussed in [chapter 20](#).

Primitive specializations and generics

In Java, all primitive types (`int`, for example) have a corresponding object type (here, `java.lang.Integer`). Often, programmers refer to these types as unboxed and boxed. Although this distinction has the laudable aim of increasing runtime efficiency, the types can become confusing. Why, for example, do you write `Predicate<Apple>` instead of `Function<Apple, Boolean>` in Java 8? An object of type `Predicate<Apple>`, when called by the `test` method, returns a primitive `boolean`.

By contrast, like all Java generics, a `Function` can be parameterized only by object types. In the case of `Function<Apple, Boolean>`, this is the object type `Boolean`, not the primitive type `boolean`. `Predicate<Apple>` is more efficient because it avoids boxing the `boolean` to make a `Boolean`. This issue has led to the creation of multiple similar interfaces such as `LongToIntFunction` and `BooleanSupplier`, which add further conceptual overload.

Another example concerns the differences between `void`, which can qualify only method return types and has no values, and the object type `Void`, which has `null` as its only value (a question that regularly appears in forums). The special cases of `Function` such as `Supplier<T>`, which could be written `() => T` in the new notation proposed in the previous section, further attest to the ramifications caused by the distinction between primitive and object types. We discussed earlier how reified generics could address many of these issues.

Other formats available



Audiobook



for immutability

have been a little upset when we said that Java 8 has three forms of values:

- Primitive values
- (References to) objects

- (References to) functions

At one level, we're going to stick to our guns and say, "But these are the values that a method may now take as arguments and return as results." But we also want to concede that this explanation is a little problematic. To what extent do you return a (mathematical) value when you return a reference to a mutable array? A `String` or an immutable array clearly is a value, but the case is far less clear-cut for a mutable object or array. Your method may return an array with its elements in ascending order, but some other code may change one of its elements later.

If you're interested in functional-style programming in Java, you need linguistic support for saying "immutable value." As noted in [chapter 18](#), the keyword `final` doesn't achieve this purpose; it only stops the field that it qualifies from being updated. Consider this example:

```
final int[] arr = {1, 2, 3};
final List<T> list = new ArrayList<>();
```

The first line forbids another assignment `arr = ...` but doesn't forbid `arr[1] = 2`; the second line forbids assignments to `list` but doesn't forbid other methods from changing the number of elements in `list`. The keyword `final` works well for primitive values, but for references to objects, it often produces a false sense of security.

What we're leading up to is this: given that functional-style programming puts strong emphasis on not mutating existing structure, a strong argument exists for a keyword such as `transitively_final`, which can qualify fields of reference type and ensure that no modification can take place in the field or any object directly or indirectly accessible via that field.

Such types represent one intuition about values: values are immutable, and only variables (which contain values) may be mutated to contain a different immutable value. As we remarked at the beginning of this sec-

Other formats available



Audiobook



ing us) sometimes inconsistently talk about the being a mutable array. In the next section, we and discuss the idea of value types, which can contain only immutable values even if variables of value types can still be updated unless they're qualified with `final`.

21.4.5. Value types

In this section, we discuss the difference between primitive types and object types, following up on the discussion of the desire for value types, which help you write programs functionally, as object types are necessary for object-oriented programming. Many of the issues we discuss are related, so there's no easy way to explain one problem in isolation. Instead, we identify the problem by its various facets.

Can't the compiler treat `Integer` and `int` identically?

Given all the implicit boxing and unboxing that Java has slowly acquired since Java 1.1, you might ask whether it's time for Java to treat, for example, `Integer` and `int` identically and to rely on the Java compiler to optimize into the best form for the JVM.

This idea is wonderful in principle, but consider the problems surrounding adding the type `Complex` to Java to see why boxing is problematic. The type `Complex`, which models so-called complex numbers with real and imaginary parts, is naturally introduced as follows:

```
class Complex {
    public final double re;
    public final double im;
    public Complex(double re, double im) {
        this.re = re;
        this.im = im;
    }
    public static Complex add(Complex a, Complex b) {
        return new Complex(a.re+b.re, a.im+b.im);
    }
}
```

But values of type `Complex` are reference types, and every operation on `Complex` needs to do an object allocation, dwarfing the cost of the two additions in `add`. Programmers need a primitive-type analog of `Complex`, perhaps called `complex`.

Other formats available



Audiobook

✗ Users want an unboxed object, for which neither
y real support. You can return to the lament
er can optimize this.” Sadly, this process is

much harder than it appears; although a compiler optimization based on
so-called escape analysis can sometimes determine that unboxing is okay,
its applicability is limited by Java's assumptions of `Objects`, which have
been present since Java 1.1. Consider the following puzzler:


```
double d1 = 3.14;
double d2 = d1;
Double o1 = d1;
Double o2 = d2;
Double ox = o1;
System.out.println(d1 == d2 ? "yes" : "no");
System.out.println(o1 == o2 ? "yes" : "no");
System.out.println(o1 == ox ? "yes" : "no");
```

The result is “yes”, “no”, “yes.” An expert Java programmer probably would say, “What silly code. Everyone knows you should use `equals` on the last two lines instead of `==`.” But we’ll persist. Even though all these primitives and objects contain the immutable value 3.14 and should be indistinguishable, the definitions of `o1` and `o2` create new objects, and the `==` operator (identity comparison) can tell them apart. Note that on primitives, the identity comparison does bitwise comparison, but on objects, it does reference equality. Often, you accidentally create a new distinct `Double` object, which the compiler needs to respect because the semantics of `Object`, from which `Double` inherits, require this. You’ve seen this discussion before, both in the discussion of value types in this chapter and in [chapter 19](#), where we discussed referential transparency of methods that functionally update persistent data structures.

Value types: Not everything is a primitive or an object

We suggest that the resolution of this problem is to rework the Java assumptions that (1) everything that isn’t a primitive is an object and hence inherits `Object` and (2) all references are references to objects.

The development starts this way. Values take two forms:

- Object types that have mutable fields unless forbidden with `final` and that also have identity, which may be tested with `==`.
- Value types, which are immutable and don’t have reference identity. Primitive types are a subset of this wider notion.

Other formats available



Audiobook



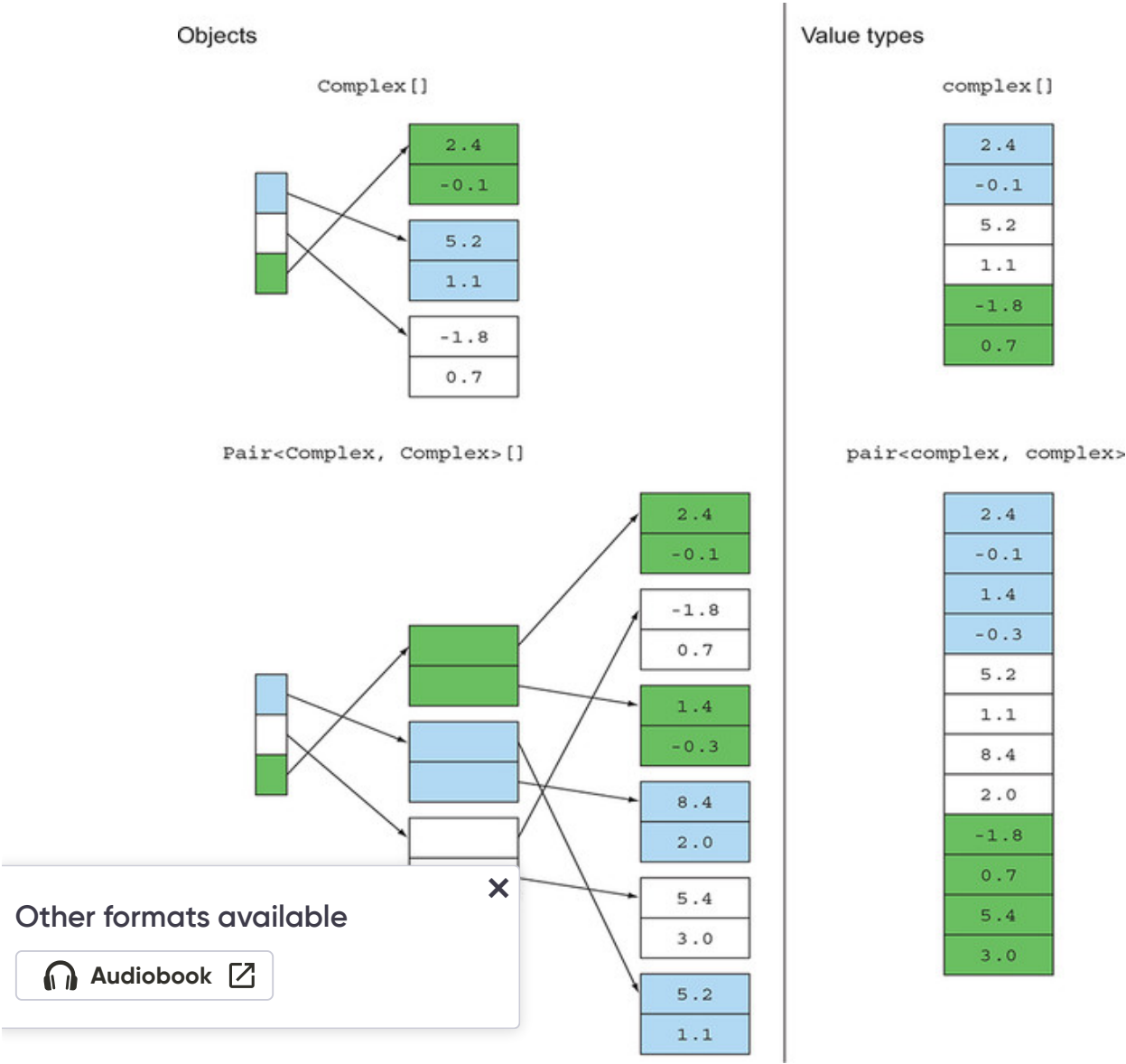
ed value types (perhaps starting with a lower-
 eir similarity to primitive types such as `int`
 pes, `==` would, by default, perform an element-

by-element comparison in the same way that hardware comparison on `int` performs a bit-by-bit comparison. We need to be careful for floating-point members as comparison is a somewhat more sophisticated opera-

tion. The type `Complex` would be a perfect example of a nonprimitive value type; such types resemble C# structs.

In addition, value types can reduce storage requirements because they don't have reference identity. **Figure 21.1** illustrates an array of size three, whose elements 0, 1, and 2 are light gray, white, and dark gray, respectively. The left diagram shows a typical storage requirement when `Pair` and `Complex` are `Objects`, and the right diagram shows the better layout when `Pair` and `Complex` are value types. Note that we call them `pair` and `complex` in lowercase in the diagram to emphasize their similarity to primitive types. Note also that value types are likely to produce better performance, not only for data access (multiple levels of pointer indirection replaced by a single indexed-addressing instruction), but also for hardware cache use (due to data contiguity).

Figure 21.1. Objects versus value types



Note that because value types don't have reference identity, the compiler can box and unbox them at its choice. If you pass a `complex` as argument

from one function to another, the compiler can naturally pass it as two separate doubles. (Returning it without boxing is trickier in the JVM, of course, because the JVM provides only method-return instructions for passing values representable in a 64-bit machine register.) But if you pass a larger value type as an argument (perhaps a large immutable array), the compiler can instead, transparently to the user, pass it as a reference when it has been boxed. Similar technology already exists in C#.

Microsoft says (<https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/value-types>):

Variables that are based on value types directly contain values. Assigning one value type variable to another copies the contained value. This differs from the assignment of reference type variables, which copies a reference to the object but not the object itself.

At the time of writing (2018), a JDK enhancement proposal is pending for value types in Java (<http://openjdk.java.net/jeps/169>).

Boxing, generics, value types: the interdependency problem

We'd like to have value types in Java because functional-style programs deal with immutable values that don't have identity. We'd like to see primitive types as a special case of value types, but the erasure model of generics, which Java currently has, means that value types can't be used with generics without boxing. Object (boxed) versions (such as Integer) of primitive types (such as int) continue to be vital for collections and Java generics because of their erasure model, but now their inheriting Object (and, hence, reference equality) is seen as a drawback. Addressing any of these problems means addressing them all.

21.5. Moving Java forward faster

There have been ten major releases of Java in 22 years—an average of more than two years between releases. In some cases, the wait was five years. The Java architects realized that this situation is no longer sustainable because it doesn't evolve the language fast enough and is the main

Other formats available



Audiobook

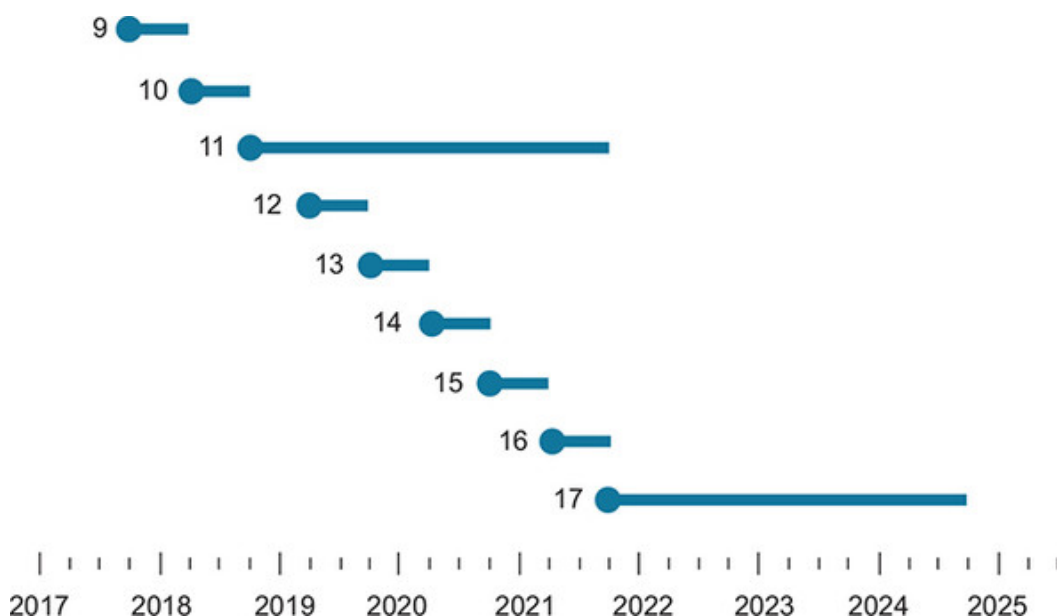


languages on the JVM (such as Scala and Kotlin) are a good model for Java. Such a long release cycle is arguably not ideal for evolutionary features such as lambdas and the Java Module System, but it also implies that minor improvements have to wait, for no valid reason, for the complete implementation of one of those big changes before being incorporated into the language. The collection

factory methods discussed in [chapter 8](#), for example, were ready to ship long before the Java 9 module system was finalized.

For these reasons, it has been decided that from now on, Java will have a six-month development cycle. In other words, a new major version of Java and the JVM will appear every six months, with Java 10 released in March 2018 and Java 11 due in September 2018. The Java architects also realized that although this faster development cycle is beneficial for the language itself, and also for agile companies and developers who are used to constantly experimenting with new technologies, it could be problematic for more conservative organizations, which generally update their software at a slower pace. For that reason, the Java architects also decided that every three years, there'll be a long-term support (LTS) release that will be supported for the subsequent three years. Java 9 isn't an LTS release, so it's considered to be at the end of its life now Java 10 is out. The same thing will happen with Java 10. Java 11, by contrast, will be an LTS version, with release planned for September 2018 and supported until September 2021. [Figure 21.2](#) shows the life cycle of the Java versions that are planned for released in the next few years.

Figure 21.2. The life cycle of future Java releases



We strongly empathize with the decision to give Java a shorter develop-

Other formats available

 Audiobook 

× adays, when all software systems and lan-
ve as quickly as possible. A shorter develop-
evolve at the right speed and allows the lan-

guage to remain relevant and appropriate in the coming years.

21.6. The final word

This book explored the main new features added by Java 8 and 9. Java 8 represents perhaps the biggest evolution step ever taken by Java. The only comparably large evolution step was the introduction, a decade previously (in 2005), of generics in Java 5. The most characteristic feature of Java 9 is the introduction of the long-awaited module system, which is likely to be more interesting for software architects than to developers. Java 9 also embraced reactive streams by standardizing its protocol through the Flow API. Java 10 introduces local-variable type inference, which is a popular feature in other programming languages to help productivity. Java 11 allows the `var` syntax of local-variable type inference to be used in the list of parameters of an implicitly typed lambda expression. Perhaps more importantly, Java 11 embraces the concurrency and reactive programming ideas discussed in this book and brings a new asynchronous HTTP client library that fully adopts `CompletableFuture`. Finally, at the time of writing, Java 12 was announced to support an enhanced `switch` construct that can be used as an expression instead of just a statement—a key feature of functional programming languages. In fact, `switch` expressions pave the way for the introduction of pattern matching in Java, which we discussed in [section 21.4.2](#). All these language updates show that functional programming ideas and influence will continue to make their way into Java in the future!

In this chapter, we looked at pressures for further Java evolution. In conclusion, we propose the following statement:

Java 8, 9, 10, and 11 are excellent places to pause but not to stop!

We hope that you’ve enjoyed this learning adventure with us and that we’ve sparked your interest in exploring the further evolution of Java.

Other formats available



Audiobook

